```
  moveUp() {
    if (this.isFirst()) {
      return
    }
    this.project.swapTasksAt(this.index() - 1, this.index())
    this.updater.update("up")
  }

  moveDown() {
    if (this.isLast()) {
      return
    }
    this.project.swapTasksAt(this.index(), this.index() + 1)
    this.updater.update("down")
  }
}
```

And here's the code for TestUpdater:

js_jasmine/03/app/javascript/packs/task_updater.js

```
export class TaskUpdater {
  constructor(task) {
    this.task = task
  }

  update(upOrDown) {
    const url = `/tasks/${this.task.id}/${upOrDown}.json`
    $.ajax({
      url,
      beforeSend: xhr => {
        xhr.setRequestHeader(
          "X-CSRF-Token",
          $('meta[name="csrf-token"]').attr("content")
        )
      },
      data: { _method: "PATCH" },
      type: "POST"
    })
  }
}
```

You could unit-test the URL here the way you did before, but it's covered by the Capybara integration test, so I'm not overly concerned about it.

At this point there's some JavaScript generating HTML in the files app/javascript/packs/project_table.js and app/javascript/packs/task_row.js, but I won't go into them in depth. I used jQuery to build up the HTML, which is a bit complicated-looking. In a real app I'd use a framework or a template tool, but I'm reluctant to bring in another tool here, especially one that's not particularly relevant to testing. In any case, the HTML is also covered by the Capybara integration test.